Hadoop, HBase

Gombos Gergő

source: simplilearn

Hadoop

- Adatelemzés, elemzési platformok
  - Distributed Computing (MapReduce)
- Adattárolási technikák
  - Distributed Storage (HDFS)

Kiknek jó a hadoop?

"... to create building blocks for programmers who just happen to have ***lots of data to store, lots of data to analyze, or lots of machines to coordinate***, and who don't have the time, the skill, or the inclination to become distributed systems experts to build the infrastructure to handle it."

Tom White

Hadoop: The Definitive Guide

Kik haszálják?

Hadoop történelem

- 2004 – Google publikálja a MapReduce technikát
- 2006 – Apache projekt lett Yahoo! támogatással
- To

Hadoop Sorts a Petabyte in 16.25 Hours and a Terabyte in 62 Seconds

By aanand – Mon, May 11, 2009 11:00 AM EDT

Recommend 0 Tweet 3

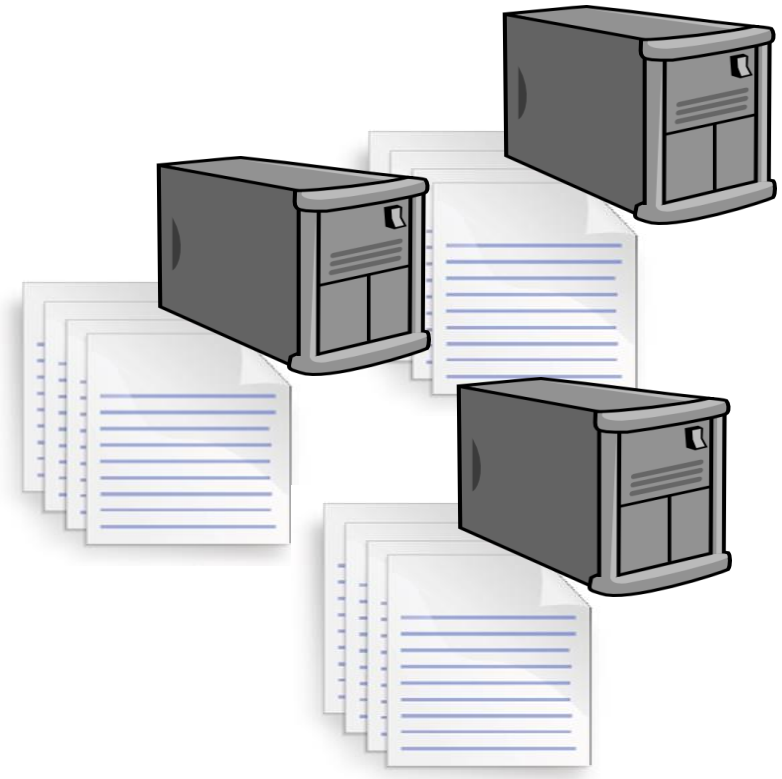
We used Apache Hadoop to compete in Jim Gray's Sort benchmark. Jim's Gray's sort benchmark consists of a set of many related benchmarks, each with their own rules. All of the sort benchmarks measure the time to sort different numbers of 100 byte records. The first 10 bytes of each record is the key and the rest is the value. The **minute sort** must finish end to end in less than a minute. The **Gray sort** must sort more than 100 terabytes and must run for at least an hour. The best times we observed were:

Bytes	Nodes	Maps	Reduces	Replication	Time
500,000,000,000	1406	8000	2600	1	59 seconds
1,000,000,000,000	1460	8000	2700	1	62 seconds
100,000,000,000,000	3452	190,000	10,000	2	173 minutes
1,000,000,000,000,000	3658	80,000	20,000	2	975 minutes

approximately 3800 nodes

HDFS

- Hadoop Distributed Filesystem (HDFS)
- Default replica szám: 3
- Replica elhelyezés:
  - 1 mindig egy másik rackben
- 2 komponens:
  - Namenode: metaadat manager
  - Datanode: tárolja az adatot
- Skálázható, új gépek hozzá vételével
- Unix-szerű-en tudjuk használni



HDFS példa

hadoop@dbpc62:~$ hdfs dfs -ls /user/ggombos/giraphInput

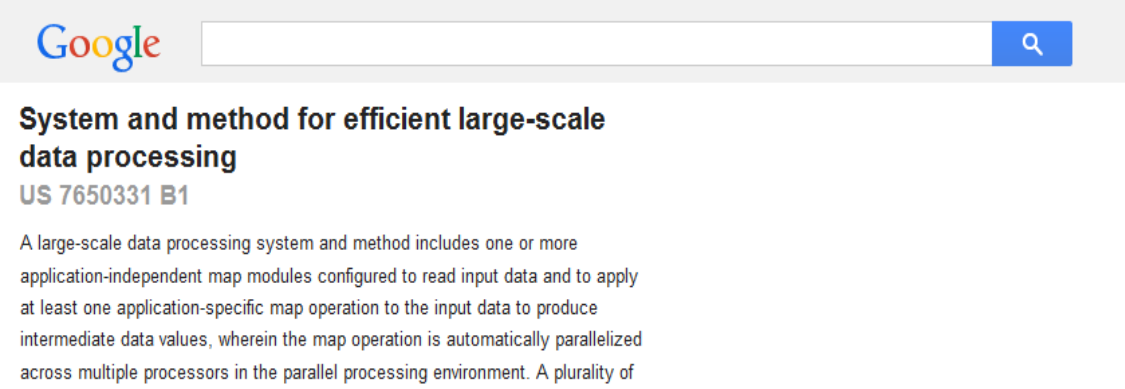
```
-rw-r--r--   3 ggombos ggombos        112 2015-01-29 15:55 /user/ggombos/giraphInput/tiny_giraph.txt
```

hadoop@dbpc62:~$ hdfs dfs -cat /user/ggombos/giraphInput/tiny_giraph.txt

```
[0,0,[[1,1],[3,3]]]
[1,0,[[0,1],[2,2],[3,1]]]
[2,0,[[1,2],[4,4]]]
[3,0,[[0,3],[1,1],[4,4]]]
[4,0,[[3,4],[2,4]]]
```

hadoop@dbpc62:~$ hdfs dfs -mkdir KorszeruDemo
hadoop@dbpc62:~$ hdfs dfs -rmdir KorszeruDemo
hadoop@dbpc62:~$ hdfs dfs -put valami.txt KorszeruDemo
hadoop@dbpc62:~$ hdfs dfs -get KorszeruDemo/valami.txt .

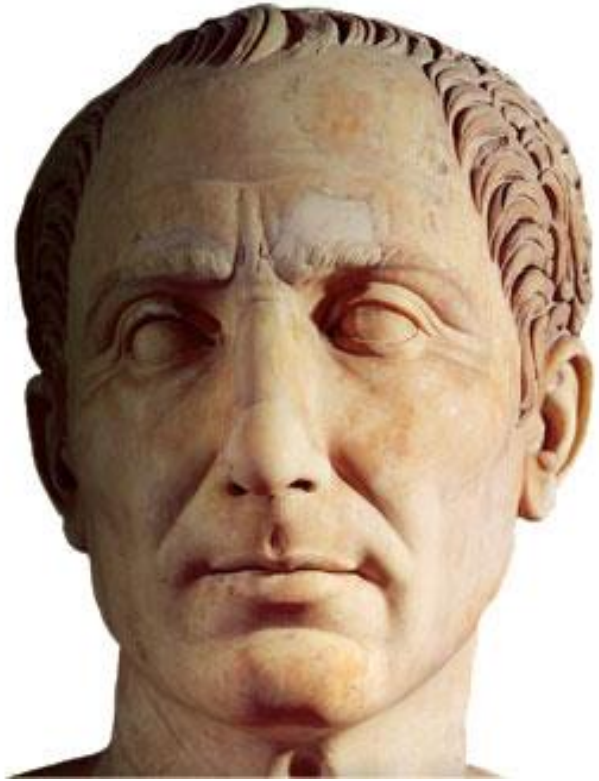
Distributed Computing



Legismertebb algoritmus, a MapReduce algoritmus, amit a Google talált ki.

DE, Google előtt is feltalálták: Julius Caesar

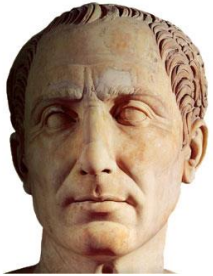
Divide et impera*

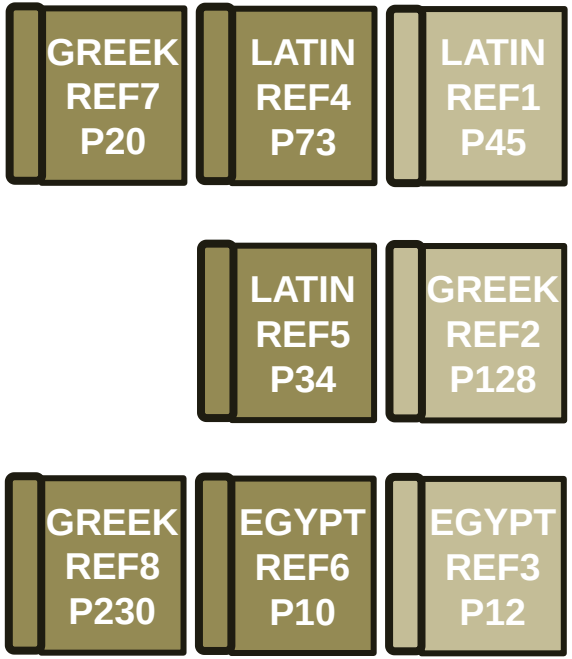


* Divide and conquer

Példa

Hány oldal íródott latin nyelven az ókori
'ria könyvtárában?





LATIN ✓
Oldal 45

Vár
Olvas..

EGYPTIAN ✗

Mappers

Reducer

Példa

Hány oldal íródott latin nyelven az ókori

ria könyvtárában?

vár

olvas…

GREEK ✗

Reducer

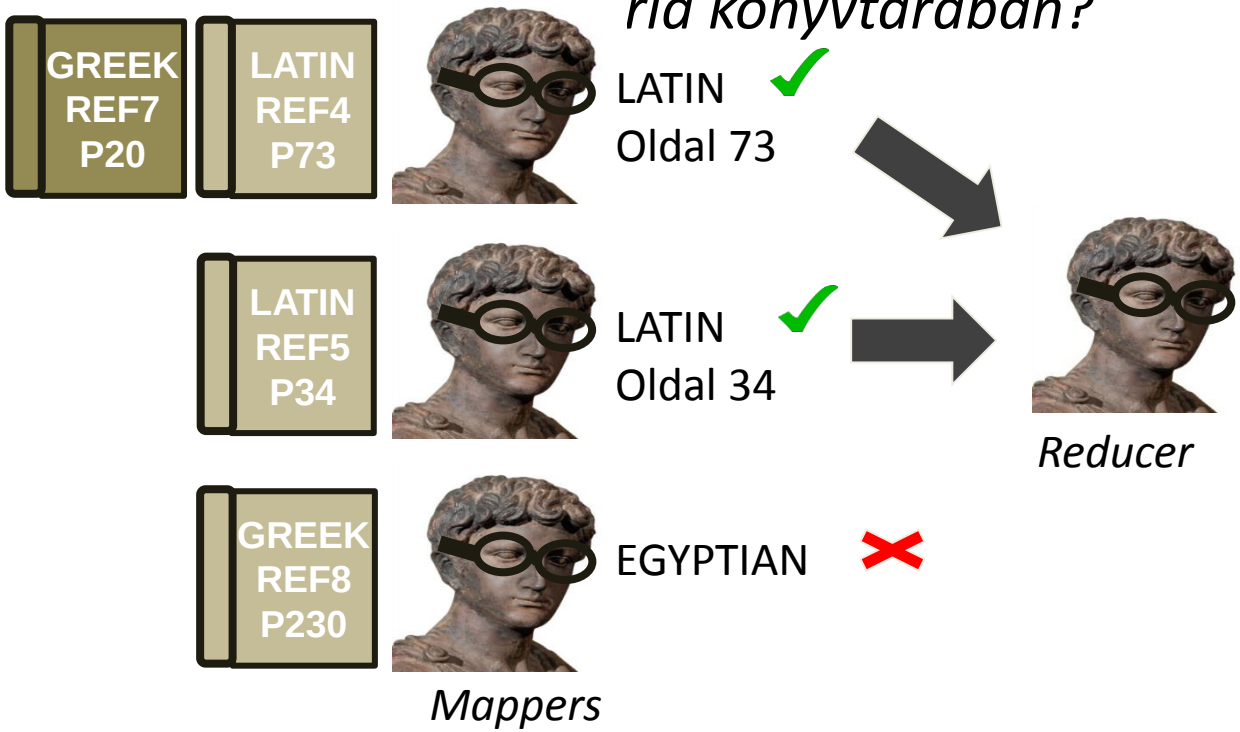
EGYPTIAN ✗

Mappers

Példa

Hány oldal íródott latin nyelven az ókori

ria könyvtárában?



Példa

Hány oldal íródott latin nyelven az ókori

ria könyvtárában?

GREEK ✗

vár…

GREEK ✗

Mappers

Reducer

Példa

Hány oldal íródott latin nyelven az ókori
ria könyvtárában?

vár…

vár…

vár…

Mappers

Reducer

Hadoop / MapReduce tulajdonságok

- Ott fusson a kód, ahol az adat van!
  - Mivel nagy adatokat kezelünk, nem célszerű azokat a hálózaton mozgatni, helyette a kódot küldjük arra a node-ra,amelyik a feldolgozást végzi.
- Több méretű fájlok (1MB vs 1TB)
  - Mivel a fájlok mérete nem egy méretű, emiatt a párhozamosan futó azonos kódok nem egyszerre fognak végezni.
  - Megoldás: (SPLIT) vágjuk a fájlokat fix méretekre (block)
- Komponensek:
  - Application Master (AM), egy adott jobot felügyel.
  - Resource Manager (RM), a klaszter erőforrásait ismerii.
  - Node Manager(NM), egy adott node erőforrásait ismeri

Hadoop / MapReduce tulajdonságok

- Hibakezelés:
  - Minden komopnens heartbeateket küldözget. Ha nem érkezik heartbeat, akkor az adott komponense / node kiesett
  - Kiesik egy NM.
    - RM szól a megfelelő AM-nek, aki megteszi a megfelelő lépéseket.
  - Kiesik az AM.
    - Az RM újraindítja az AM-t, aki szinkronizál a NM-kkel.
  - Kiesik a RM.
    - Secondary RM, aki átveszi szerepét. Szinkronban vannak.
  - Leáll egy Task futása.
    - AM újraindítja.

Hadoop / MapReduce tulajdonságok

- Task
  - Egy adott node-n lévő kód egy adott blockon. (map / reduce)
  - A replica miatt többen is tudják olvasni ugyanazt
- Speculative execute
  - Több replica olvassa ugyanazt a block-ot, és a legyorsabb válaszol, a többit leállítjuk.
- Schedulerek:
  - Több alkalmazás is futhat a rendszerben.
  - Ütemezők kezelik ezek indítását. (Fair, Capacity)

Verziók

Version	Release date
3.2.2	2021 Jan 9
2.10.1	2020 Sep 21
3.1.4	2020 Aug 3
3.3.0	2020 Jul 14

1.3.5

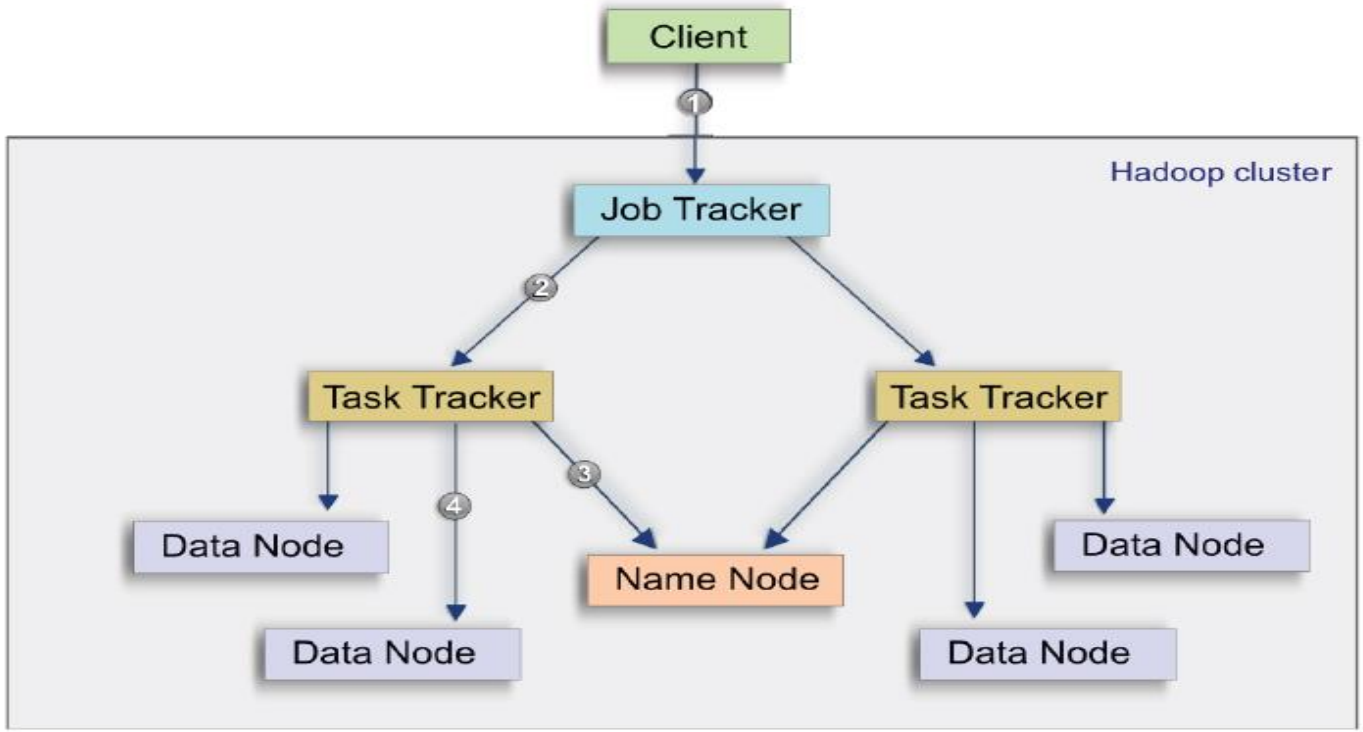
X.Y.Z

Major Version
Incremented whenever major changes are made like architectural change.

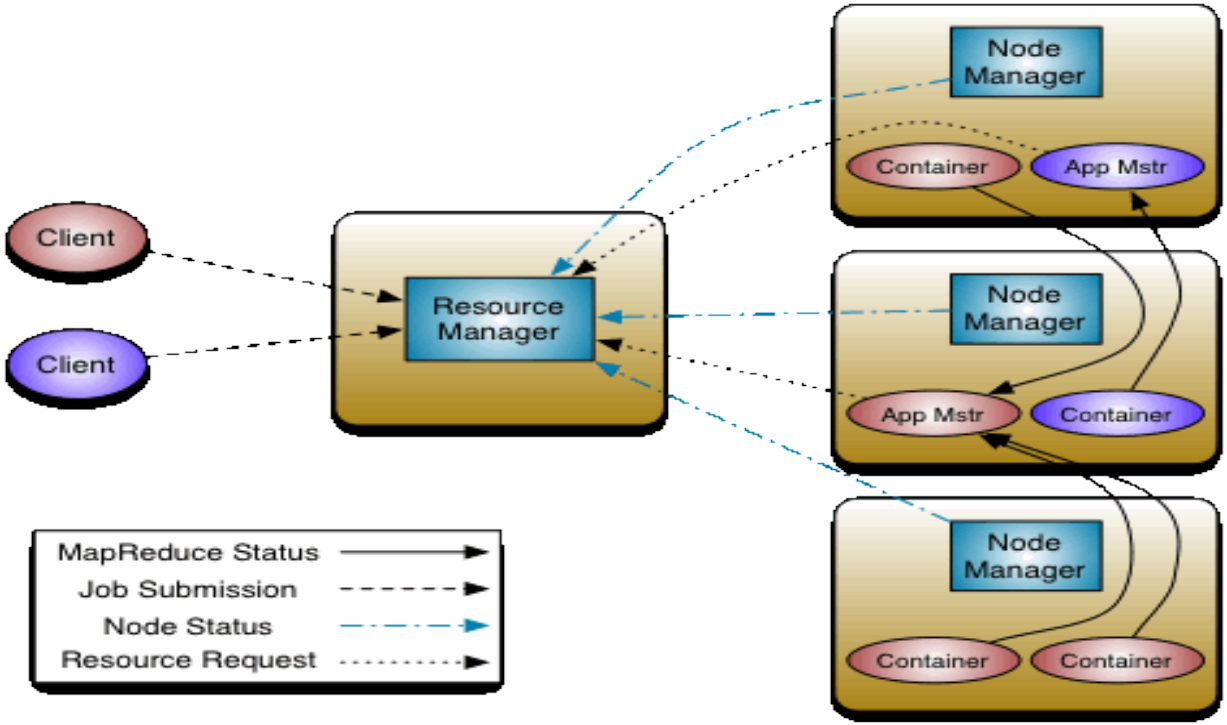
Minor Version
Incremented whenever minor changes are made which doesnot breaks the API like new feature

Patch Version
Incremented with bug fixes

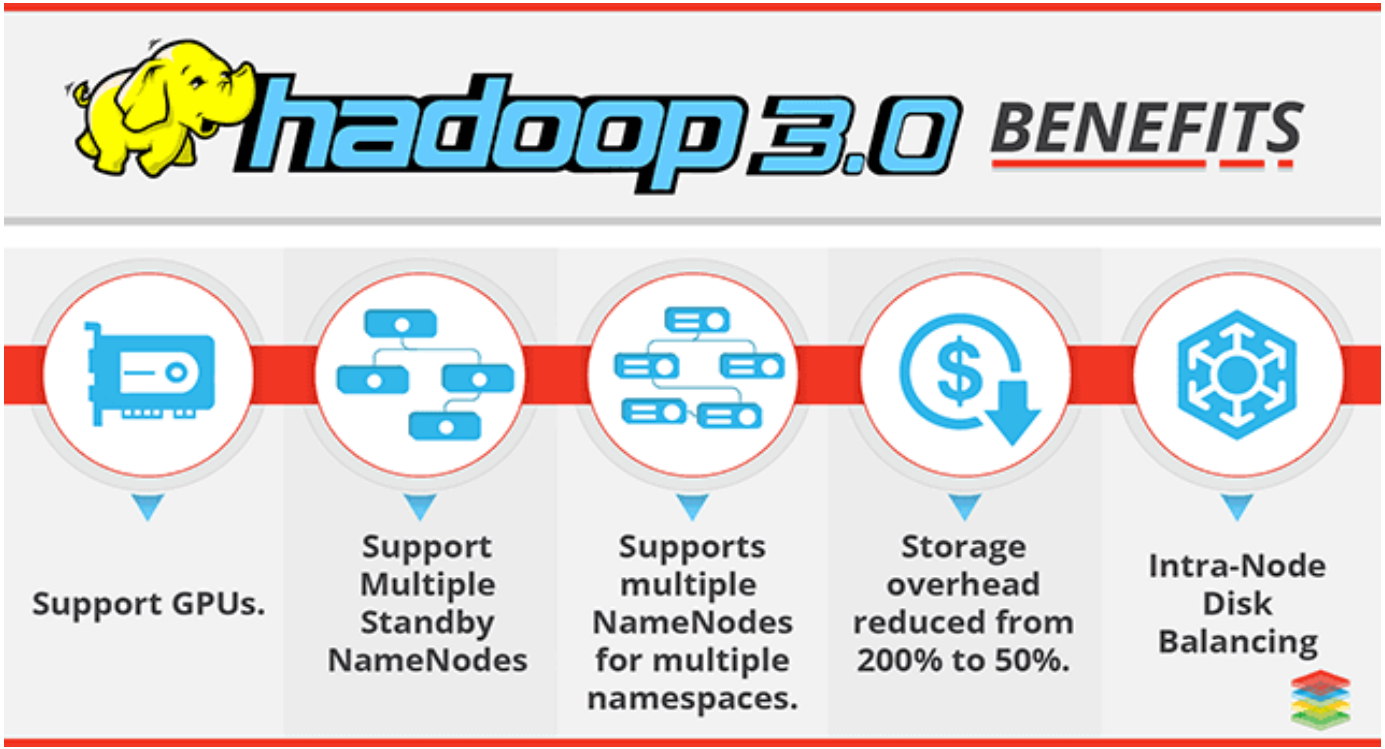
Architektúra (Hadoop 1.x)



Architektúra (Hadoop 2.x)



Hadoop 3.x


hadoop 3.0 BENEFITS
Support GPUs.
Support Multiple Standby NameNodes
Supports multiple NameNodes for multiple namespaces.
Storage overhead reduced from 200% to 50%.
Intra-Node Disk Balancing

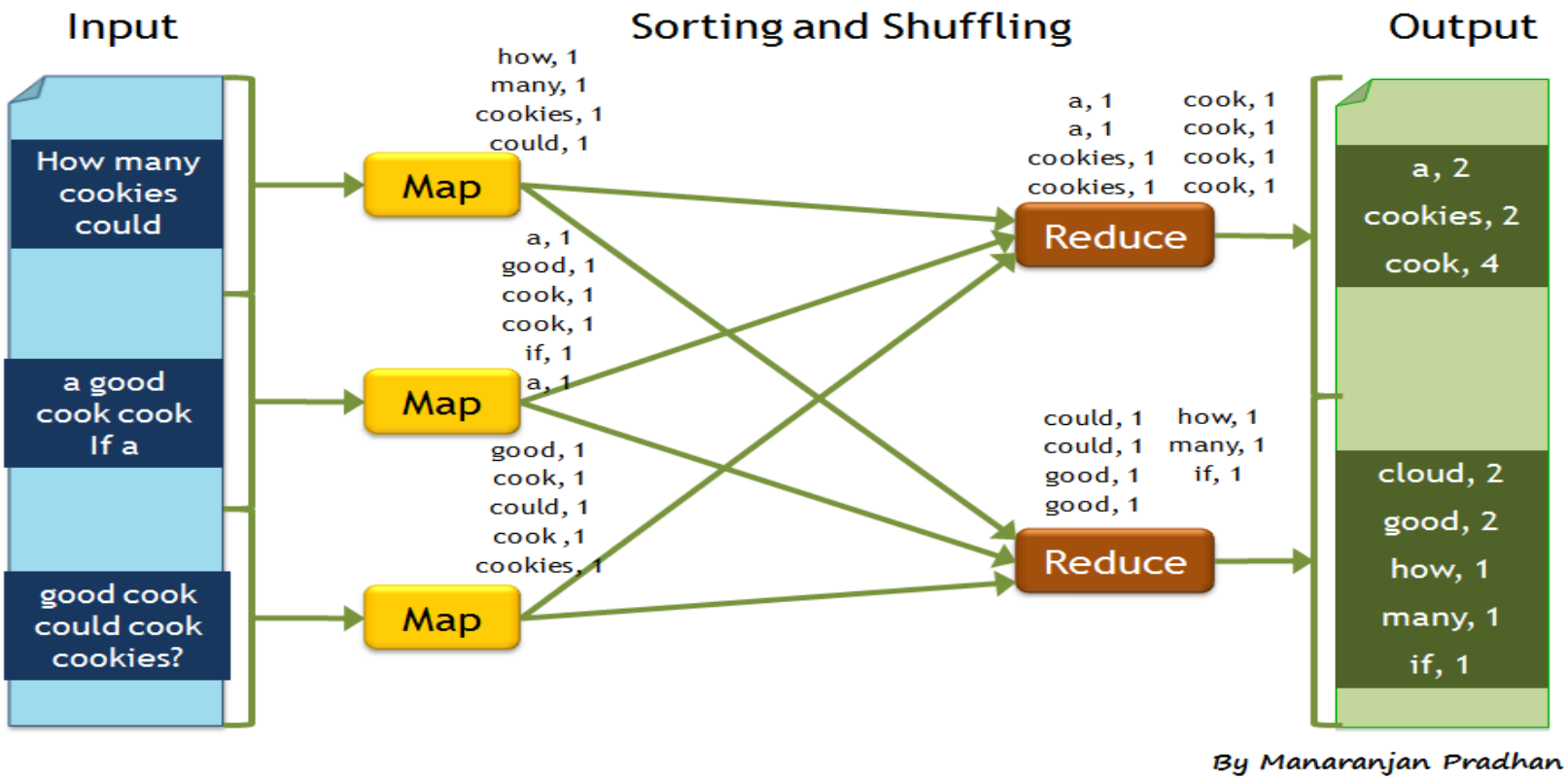
Hadoop

- Hadoop 1.0
  - MapReduce futtatás
- Hadoop 2.0
  - Yarn (Yet Another Resource Negotiator)
  - Többféle alkalmazás futtatása
- Lokalitási szabály a task futására:
  1. Ott fut ahol az adat van
  2. Abban a rackben fut ahol az adat van
  3. Bárhol

Hadoop alprojektek

- Ambari (Hadoop menedzser)
- Avro (adat formátumok támogatása)
- Cassandra (adatbázis)
- Chukwa (monitorozó)
- Hama (analizáló eszköz)
- **Hbase** (adatbázis)
- Hive (SQL szerű nyelv)
- Mahout (gépi tanulás csomag)
- Pig (lekérdező nyelv)
- Spark (gyors, memória alapú feldolgozó)
- ZooKeeper (koordinátor)

Példa


Input
Sorting and Shuffling
Output
How many cookies could
a good cook cook If a
good cook could cook cookies?
Map
Map
Map
how, 1
many, 1
cookies, 1
could, 1
a, 1
good, 1
cook, 1
cook, 1
if, 1
a, 1
good, 1
cook, 1
could, 1
cook ,1
cookies, 1
Reduce
Reduce
a, 1
a, 1
cookies, 1
cookies, 1
cook, 1
cook, 1
cook, 1
cook, 1
could, 1
could, 1
good, 1
good, 1
how, 1
many, 1
if, 1
a, 2
cookies, 2
cook, 4
cloud, 2
good, 2
how, 1
many, 1
if, 1
By Manaranjan Pradhan

MapReduce alkalmazás

- Tipikusan Java-ba íródnak.
  - Hadoop Streaming-nél lehet más nyelv.
- Futtatás:
  - Yarn jar <jar file> <main-class> <input> <output>
- Alkalmazás felépítése
  - Driver – input, output típusának definiálása, Mapper, Reducer osztály megadása
  - Mapper – Inputot dolgozza fel.
  - Reducer – A Mapper eredményét dolgozza fel.

Példa: Mapper.java

```
public static class DemoMapper extends
Mapper<Object, Text, Text, IntWritable> {

   private final Text globalKey = new Text(”num_pages");
   private final IntWritable bookPages = new IntWritable();

   @Override
   public void map(Object key, Text value, Context context)
   throws Exception {
       String[] fields = value.toString().split(“,”);

       if (fields[1].equals(“latin”)) {
         bookPages.set(fields[2]);
          context.write(globalKey, bookPages);
       }
   }

}
```

Példa: Reducer.java

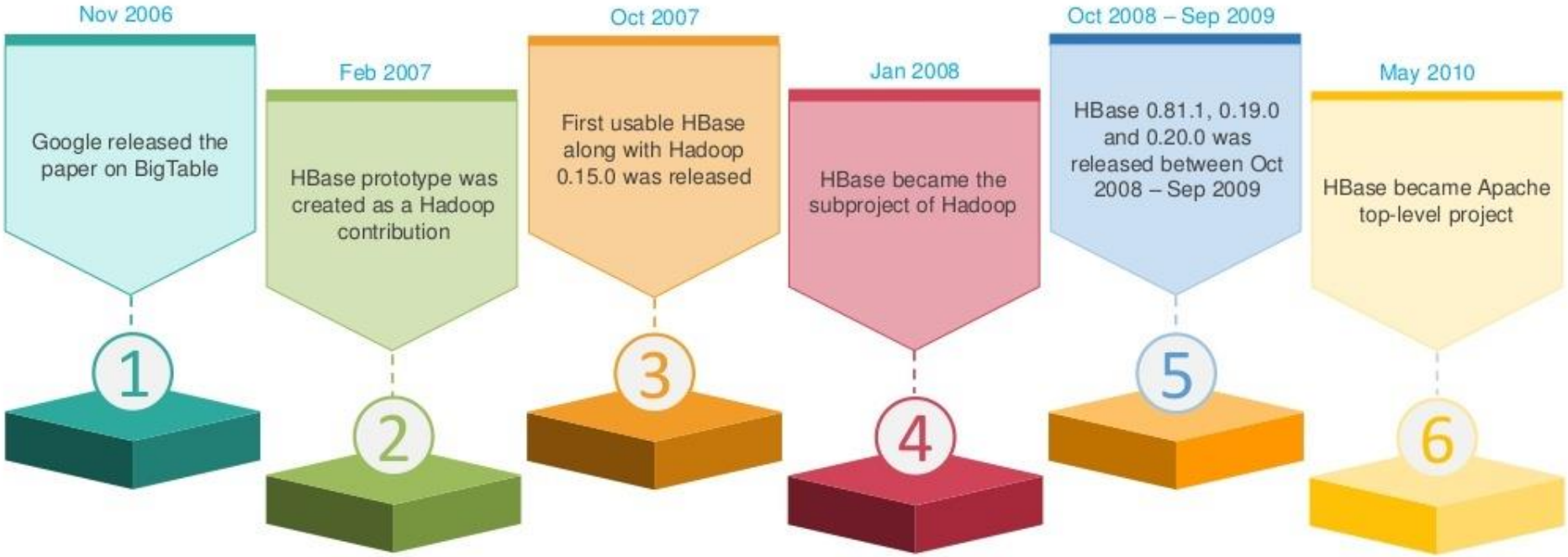
```
public static class DemoReducer extends
Reducer<Text, IntWritable, Text, IntWritable> {

   private final IntWritable totalPages= new IntWritable();

   @Override
   public void reduce(Text globalKey, Iterable<IntWritable>
   bookPages, Context context) throws Exception {
        int sum = 0;

      for (IntWritable val: bookPages) {
          sum += val.get();
        } // for

      totalPages.set(sum);
        context.write(globalKey, totalPages);
   } // reduce

}
```

Péda: DemoDriver.class

```
public class DemoDriver extends Configured implements Tool {
    public static void main(String[] args) throws Exception {
        int res = ToolRunner.run(new Configuration(), new
                  DemoDriver(), args);
        System.exit(res);
    }
      @Override
      public int run(String[] args) throws Exception {
                    Configuration conf = this.getConf();
                    Job job = Job.getInstance(conf, ”julius caesar");
                    job.setJarByClass(DemoDriver.class);
                    job.setMapperClass(DemoMapper.class);
                    job.setCombinerClass(DemoReducer.class);
                    job.setReducerClass(DemoReducer.class);
                    job.setOutputKeyClass(Text.class);
                    job.setOutputValueClass(IntWritable.class);
                    FileInputFormat.addInputPath(job, new Path(args[0]));
                    FileOutputFormat.setOutputPath(job, new Path(args[1]));
                    return job.waitForCompletion(true) ? 0 : 1;
      }
}
```

HBase

HBase History



©Simplilearn. All rights reserved.

What is HBase?

HBase is a column oriented database management system derived from Google's NoSQL database **BigTable** that runs on top of HDFS

APACHE HBASE

1. Open source project that is horizontally scalable
2. NoSQL database written in JAVA which performs faster querying
3. Well suited for sparse data sets (can contain missing or NA values)

simplilearn

©Simplilearn. All rights reserved.

Companies using HBase

©Simplilearn. All rights reserved.

HBase Use Case

Telecommunication company that provides mobile voice and multimedia services across China

Generated billions of Call Detail Records (CDR)

Traditional database systems were unable to scale up to the vast volumes of data and provide a cost-effective solution

©Simplilearn. All rights reserved.

HBase Use Case

Telecommunication company that provides mobile voice and multimedia services across China

中国移动
China Mobile

Generated billions of Call Detail Records (CDR)

Storing and real-time analysis of billions of call records was a major problem

©Simplilearn. All rights reserved.

HBase Use Case

Telecommunication company that provides mobile voice and multimedia services across China

Generated billions of Call Detail Records (CDR)

Solution

HBase stores billions of rows of detailed call records

©Simplilearn. All rights reserved.

HBase Use Case

Telecommunication company that provides mobile voice and multimedia services across China

Generates billions of Call Detail Records (CDR)

HBase performs fast processing of records using SQL queries

©Simplilearn. All rights reserved.

Applications of HBase

Medical

HBase is used for storing genome sequences

Storing disease history of people or an area

E-Commerce

HBase is used for storing logs about customer search history

Performs analytics and target advertisement for better business insights

Sports

HBase stores match details and history of each match

Uses this data for better prediction

©Simplilearn. All rights reserved.

HBase vs RDBMS

HBase	RDBMS
Does not have a fixed schema (schema-less). Defines only column families	Has a fixed schema which describes the structure of the tables
Works well with structured and semi-structured data	Works well with structured data
It can have de-normalized data (can contain missing or NA values)	RDBMS can store only normalized data
Built for wide tables that can be scaled horizontally	Built for thin tables that is hard to scale

simplilearn

©Simplilearn. All rights reserved.

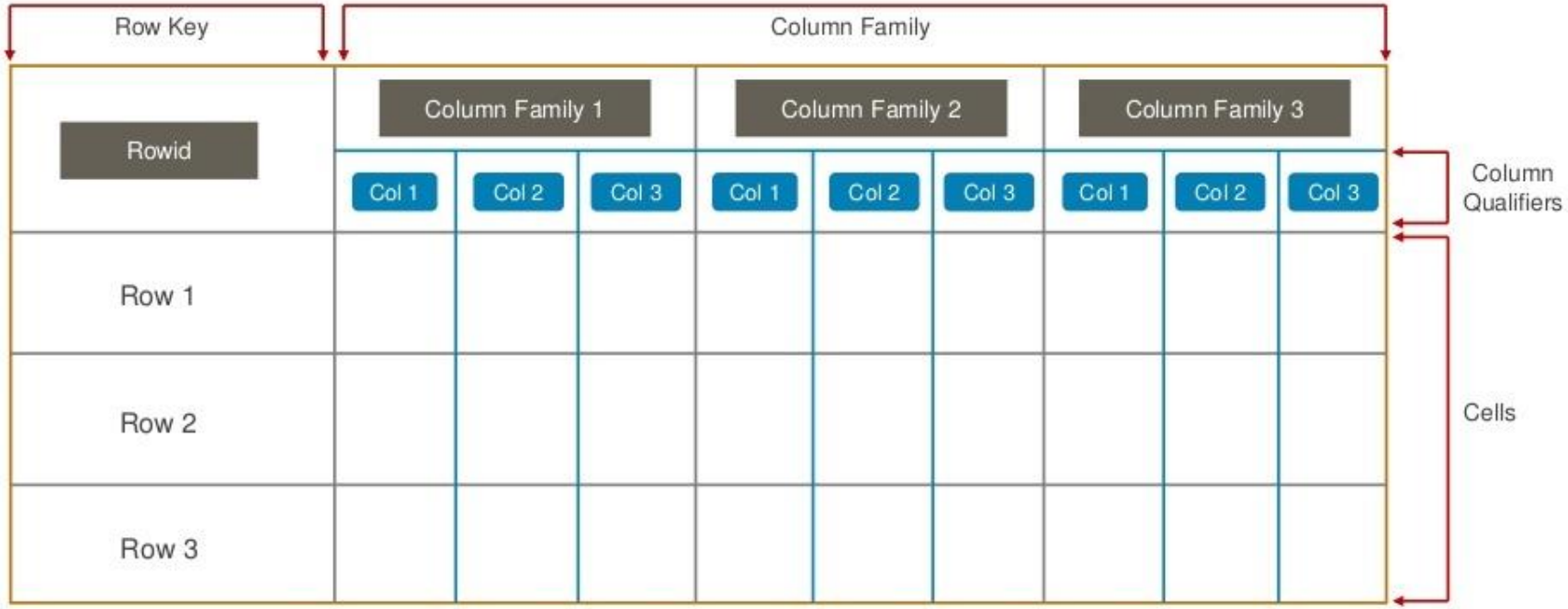
Features of HBase

Scalable	Automatic failure support	Consistent read and write	JAVA API for client access	Block cache and bloom filters
Data can be scaled across various nodes as it is stored in HDFS	Write Ahead Log across clusters which provides automatic support against failure	HBase provides consistent read and write of data	Provides easy to use JAVA API for clients	Supports block cache and bloom filters for high volume query optimization

simplilearn

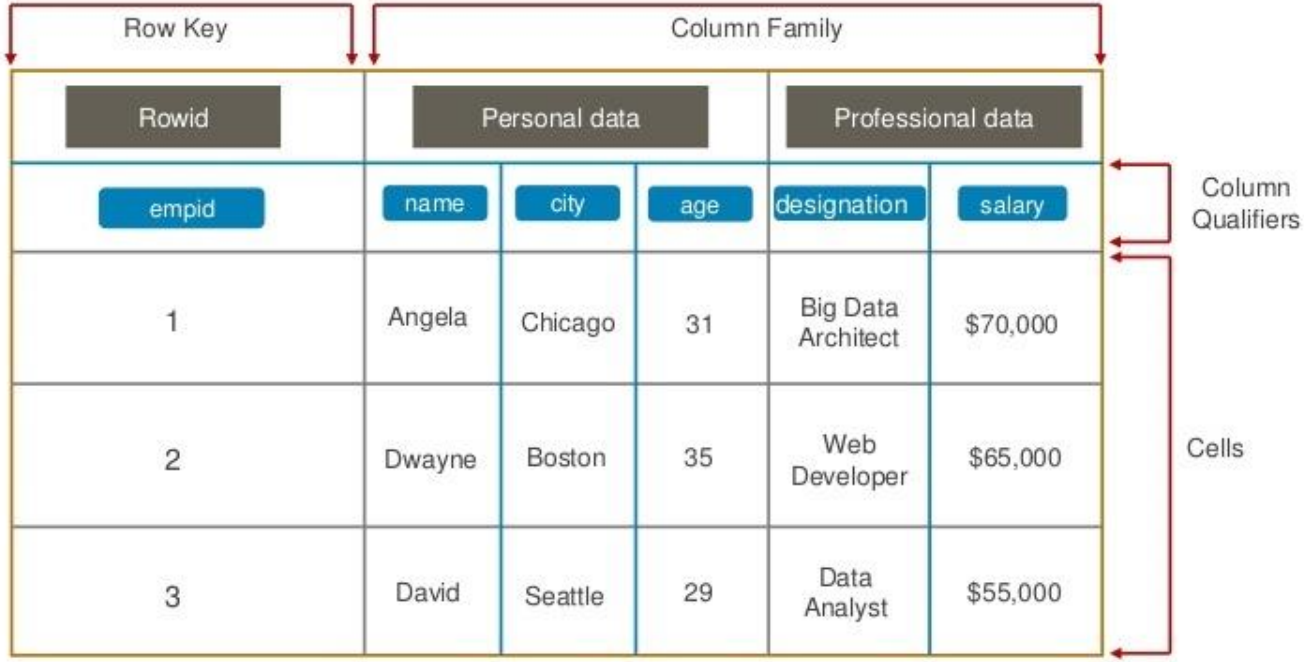
©Simplilearn. All rights reserved.

HBase column oriented storage



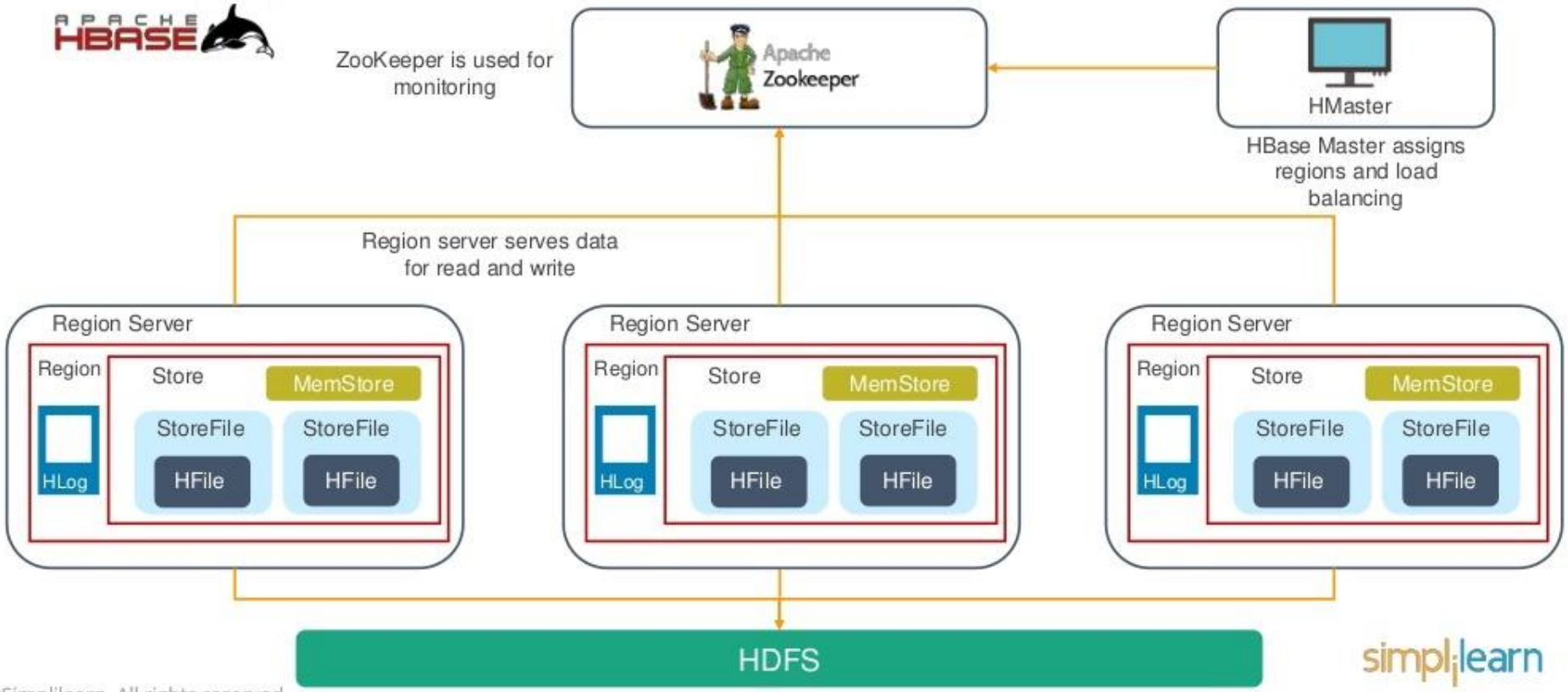
©Simplilearn. All rights reserved.

HBase column oriented storage



©Simplilearn. All rights reserved.

HBase Architectural Components



©Simplilearn. All rights reserved.

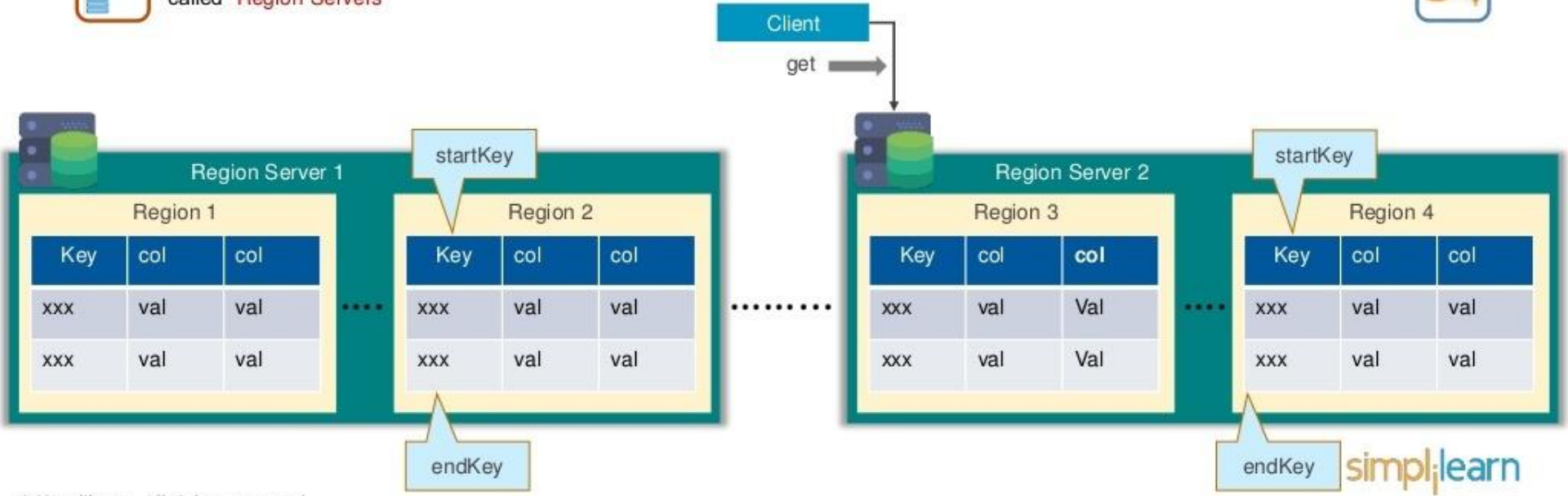
HBase Architectural Components - Regions

HBase tables are divided horizontally by row key range into "Regions"

A region contains all rows in the table between the region's start key and end key

Regions are assigned to the nodes in the cluster, called "Region Servers"

These servers serve data for read and write



©Simplilearn. All rights reserved.

HBase Architectural Components - HMaster

Region assignment, Data Definition Language operation (create, delete) are handled by HMaster

Assigning and re-assigning regions for recovery or load balancing and monitoring all servers

Client

create, delete, update table

HMaster

Monitors region servers

Assigns regions to region servers

Assigns regions to region servers

Region Server 1

Region 1

Key	col	col
xxx	val	val
xxx	val	val

Region 2

Key	col	col
xxx	val	val
xxx	val	val

Region Server 2

Region 3

Key	col	**col**
xxx	val	Val
xxx	val	Val

Region 4

Key	col	col
xxx	val	val
xxx	val	val

HBase has a distributed environment where HMaster alone is not sufficient to manage everything. Hence, ***ZooKeeper*** was introduced

©Simplilearn. All rights reserved.

HBase Architectural Components - ZooKeeper

ZooKeeper is a distributed coordination service to maintain server state in the cluster

Zookeeper maintains which servers are alive and available, and provides server failure notification

Inactive HMaster

Active HMaster

ZooKeeper

heartbeat

Region Server 1

Region 1

Key	col	col
xxx	val	val
xxx	val	val

Region 2

Key	col	col
xxx	val	val
xxx	val	val

Region Server 2

Region 3

Key	col	col
xxx	val	Val
xxx	val	Val

Region 4

Key	col	col
xxx	val	val
xxx	val	val

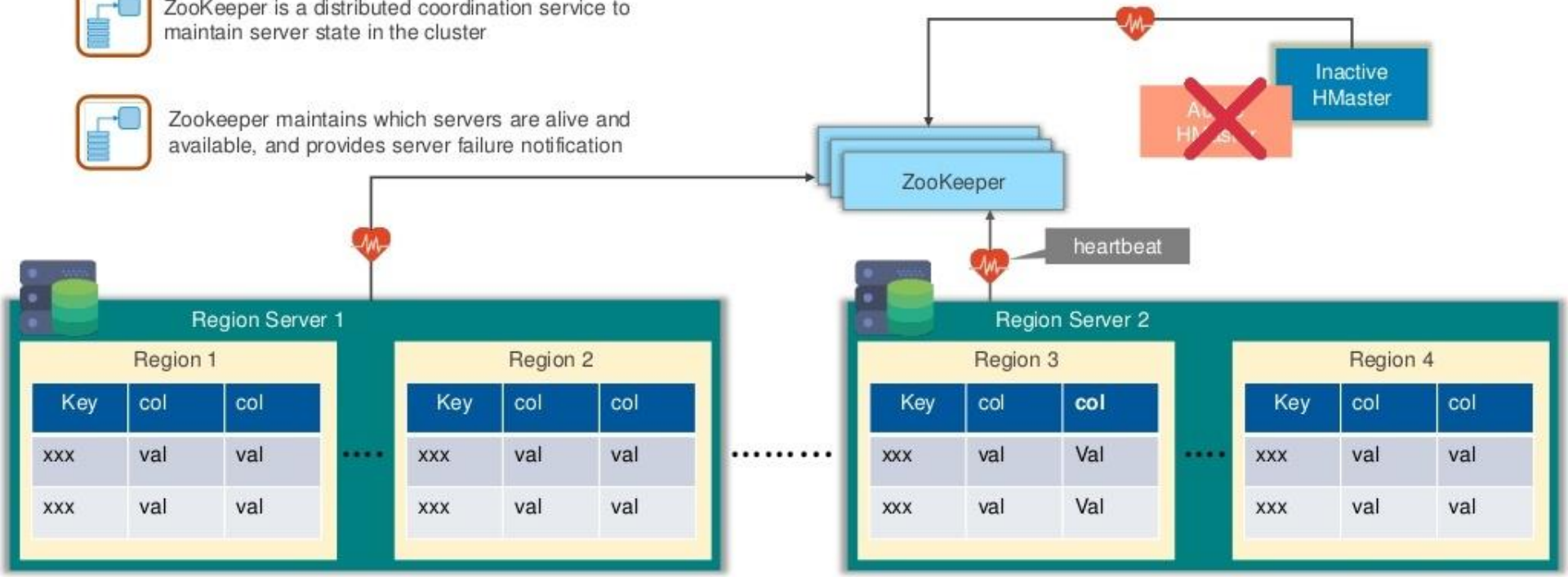
Region servers send their status to ZooKeeper indicating they are ready for read and write operation

©Simplilearn. All rights reserved.

HBase Architectural Components - ZooKeeper

ZooKeeper is a distributed coordination service to maintain server state in the cluster

Zookeeper maintains which servers are alive and available, and provides server failure notification



Inactive server acts as a backup. If the active HMaster fails, it will come to rescue

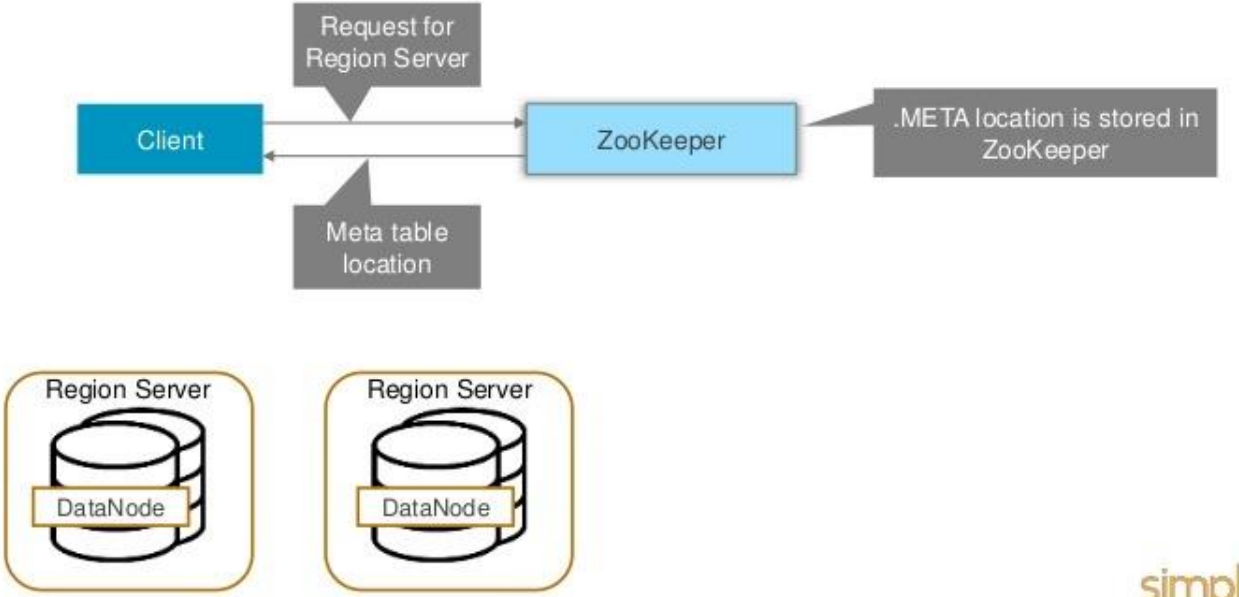
©Simplilearn. All rights reserved.

HBase Read or Write

There is a special HBase Catalog table called the META table, which holds the location of the regions in the cluster

Here is what happens the first time a client reads or writes data to HBase

The client gets the Region Server that hosts the META table from ZooKeeper



©Simplilearn. All rights reserved.

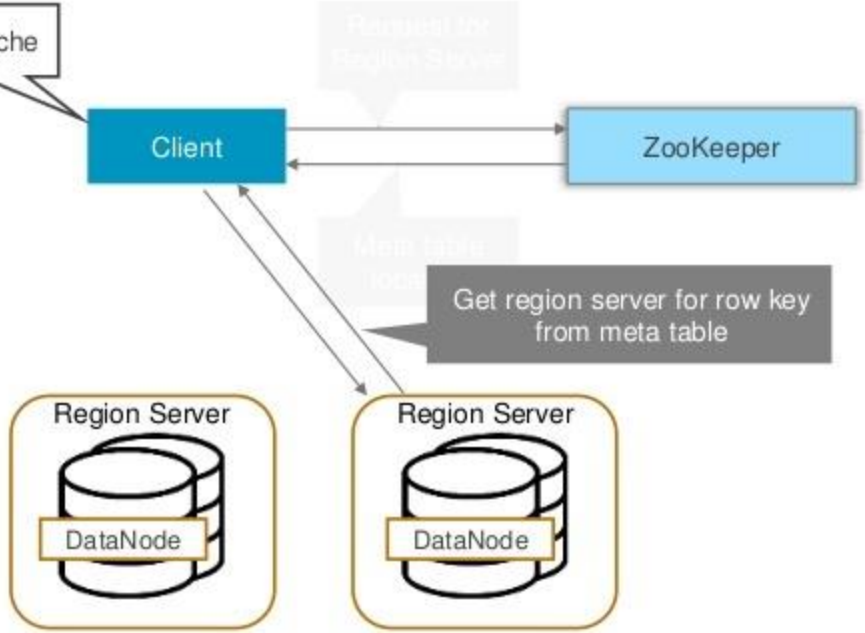
HBase Read or Write

There is a special HBase Catalog table called the META table, which holds the location of the regions in the cluster

Here is what happens the first time a client reads or writes data to HBase

The client will query the .META server to get the region server corresponding to the row key it wants to access

The client caches this information along with the META table location

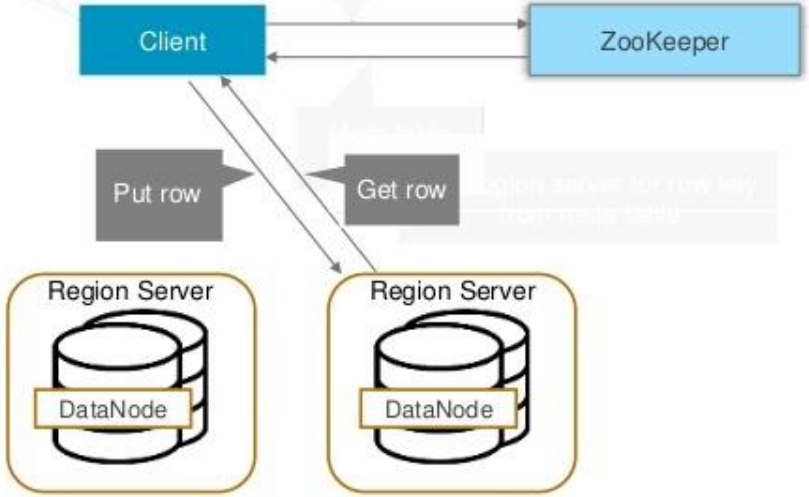


HBase Read or Write

There is a special HBase Catalog table called the META table, which holds the location of the regions in the cluster

Here is what happens the first time a client reads or writes data to HBase

It will get the Row from the corresponding Region Server



©Simplilearn. All rights reserved.

HBase Meta Table

Special HBase catalog table that maintains a list of all the Region Servers in the HBase storage system

Meta Table	
Row key	value
table, key, region	region server

META table is used to find the Region for a given Table key

Region Server

Region 1

Key	col	col
xxx	val	val
xxx	val	val

Region 2

Key	col	col
xxx	val	val
xxx	val	val

Region Server

Region 3

Key	col	col
xxx	val	val
xxx	val	val

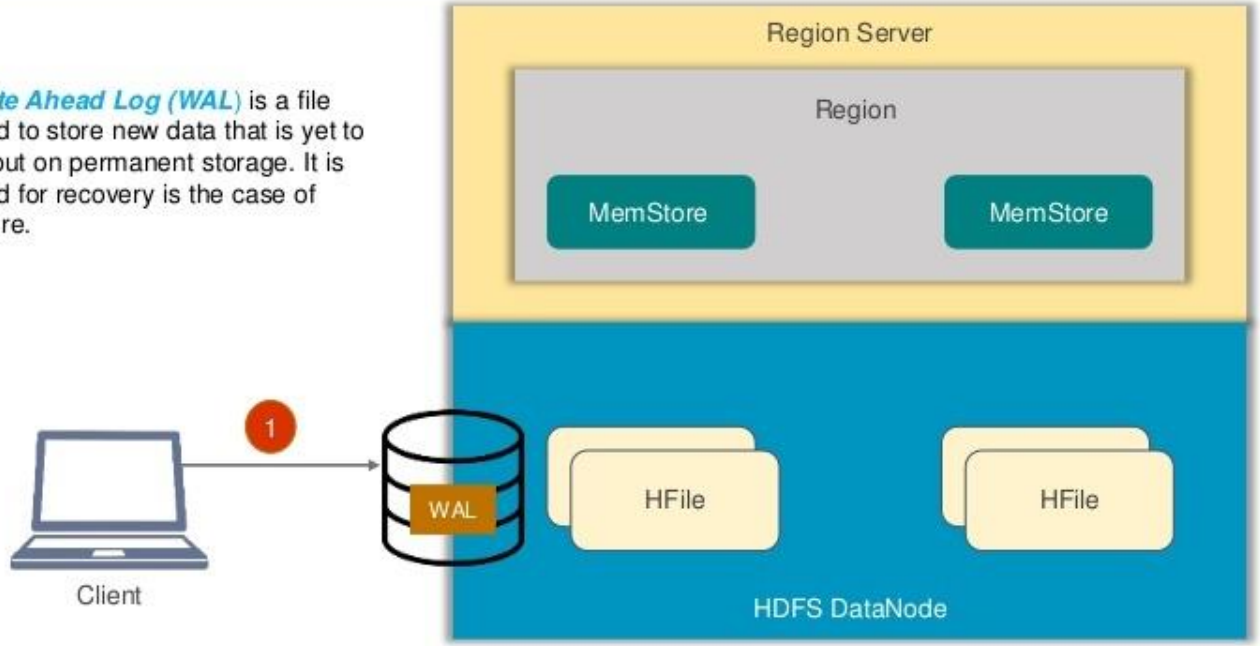
Region 4

Key	col	col
xxx	val	val
xxx	val	val

©Simplilearn. All rights reserved.

HBase Write Mechanism

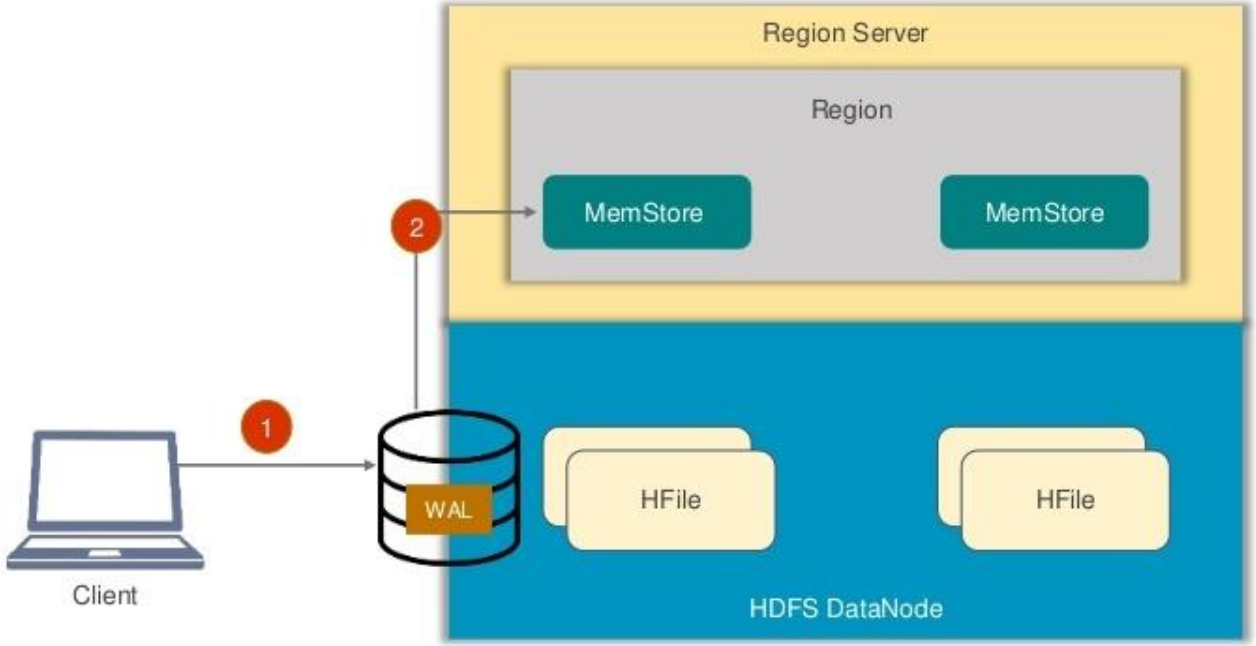
Write Ahead Log (WAL) is a file used to store new data that is yet to be put on permanent storage. It is used for recovery is the case of failure.



1 When client issues a put request, it will write the data to the write-ahead log (WAL)

©Simplilearn. All rights reserved.

HBase Write Mechanism

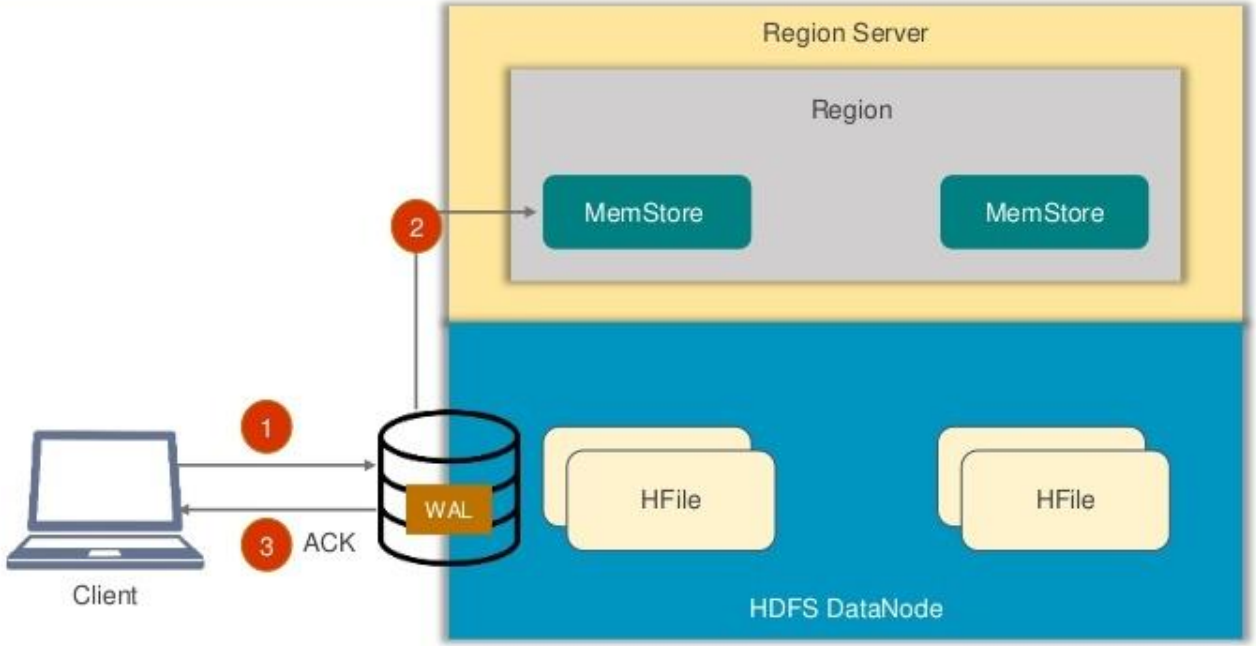


MemStore is the write cache that stores new data that has not yet been written to disk. There is one MemStore per column family per region.

2 Once data is written to the WAL, it is then copied to the MemStore

©Simplilearn. All rights reserved.

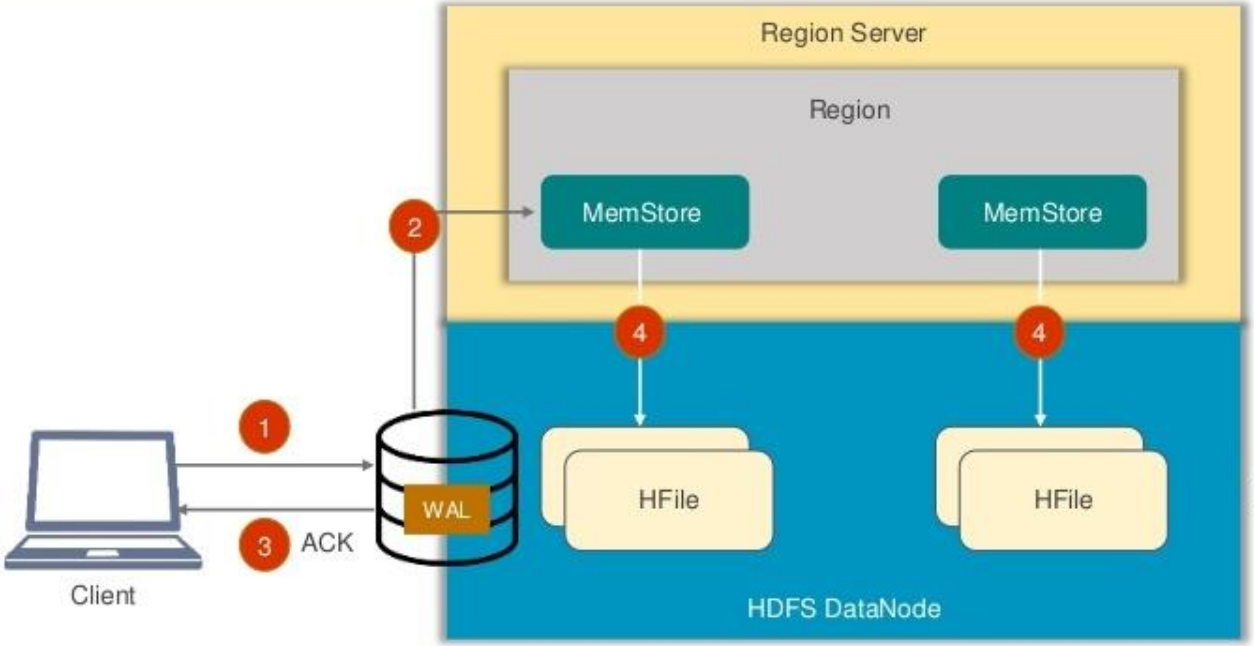
HBase Write Mechanism



3 Once the data is placed in MemStore, the client then receives the acknowledgment

©Simplilearn. All rights reserved.

HBase Write Mechanism



Hfiles store the rows of data as sorted KeyValue on disk

4 When the MemStore reaches the threshold, it dumps or commits the data into a HFile

simplilearn

©Simplilearn. All rights reserved.

Köszönöm a figyelmet!